

One of the powerful features of visualizing data using computers is that you aren't limited to a static view of your data. Users can interact with your application using their mouse and keyboard (or other modalities). This can then update how the data is presented, allowing users to explore and direct what is shown to them.

Here are some example interactions:

- Details on demand: You can't show the user everything. So, some details you can make available 'on demand', such as when a user hovers over an element.
- Filter: Perhaps you begin by showing the user an overview of the data. But, maybe you want to let them make selection which filters the data, updating the view to show the portion of the data that are interested in focusing on.
- Zoom: You can enable users to 'zoom in' or 'zoom out' of a complex scene.
- Brushing: You can create actions that allow users to 'brush' over elements in your visualization, selecting the underlying data, and then you can show more details or show the selected data in a second view.
- Coordination between multiple views: If you have multiple views of the data, you can create actions that extend between visualizations, so the presentation in one is connected to the actions users take in another view.

We will explore these kinds of interactions and more during lecture this term, but let's look at the technical side- how can you change what is shown to the user with d3?

For this tutorial we are going to be referencing this example. This example is adapted from code written by the creators of UBC's InfoVis course (<https://github.com/UBC-InfoVis/2021-436V-examples/tree/master/d3-interactive-scatter-plot>)

EXAMPLE HERE:

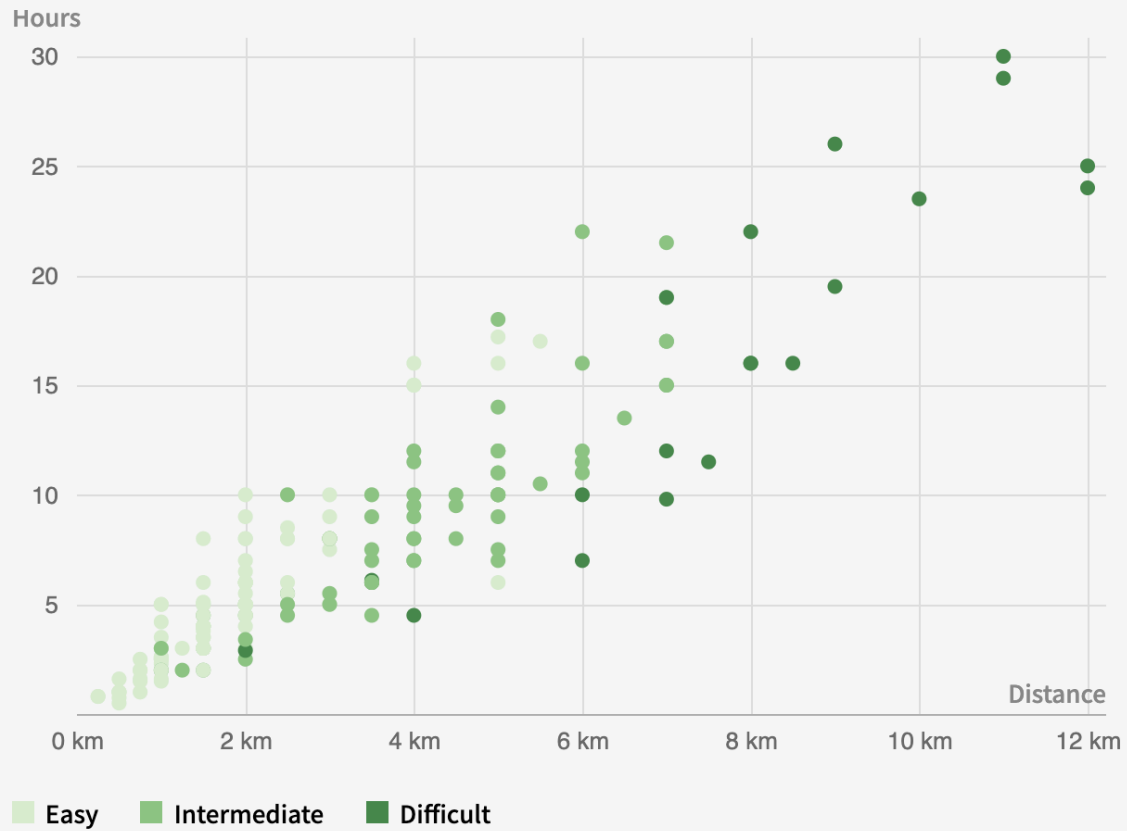
https://drive.google.com/drive/folders/1KvqtKd4TR05JogVgdGTIGwOcJV5wCaQX?usp=drive_link

1. Details on demand

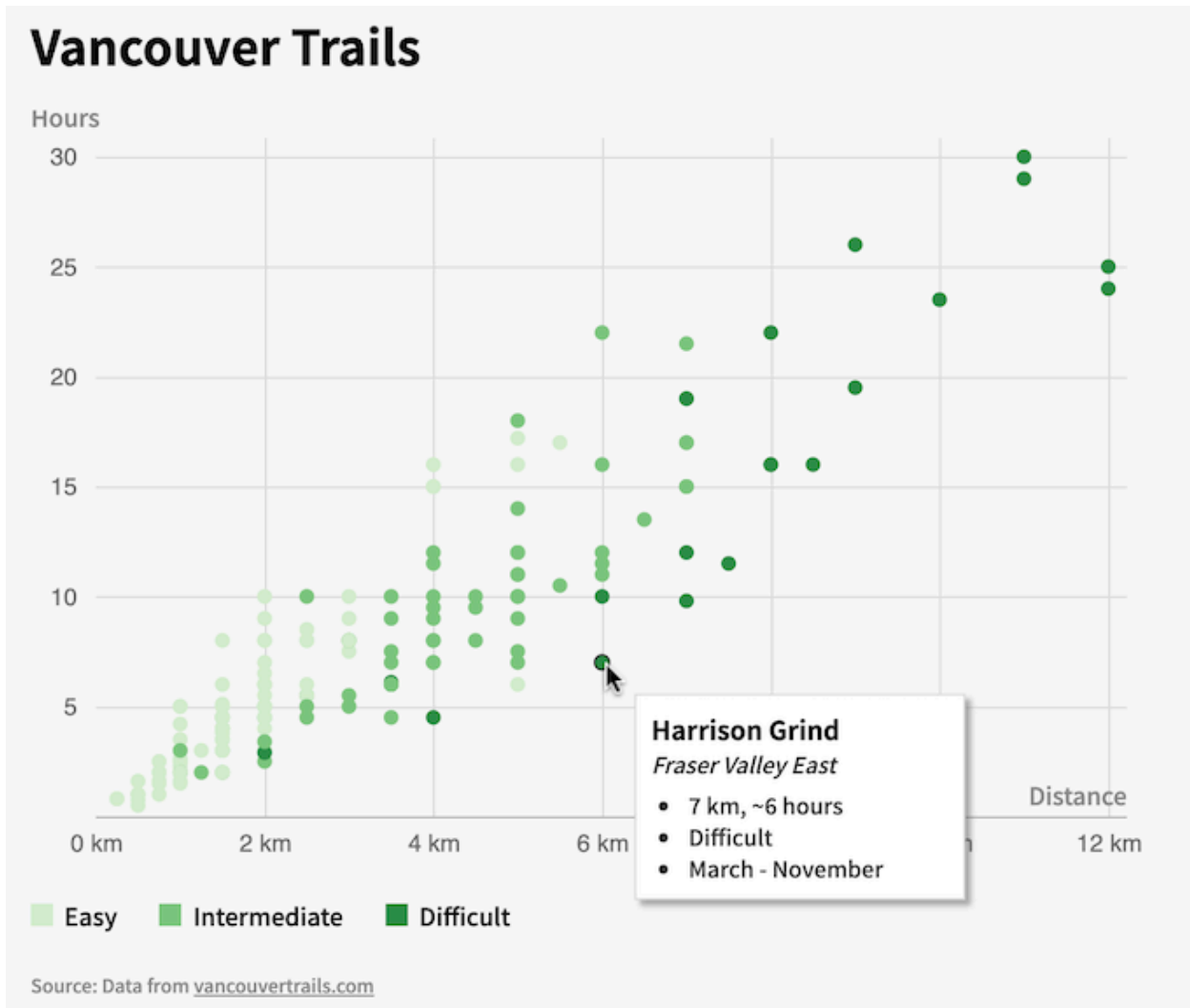
Suppose you want to give the user details about the points in a scatterplot.

Before user interacts:

Vancouver Trails



After they hover over a point in the scatterplot:



This is a 'hover'/'mouseover' action. When hovering over the point, you are giving the user information that is not always available. Imagine if you tried to make this information available for every point in your chart by default. It would be too much. Too cluttered. Visually overwhelming. Here, we make these details available 'on-demand'. The user can see the trend in the hiking trails in the scatterplot, and then they can select individual points to see details.

How does this example work?

Much of the code is the same as d3 code we have written before. First, you setup your project like usual- main.js for loading the data, and initializing the components of the view, a class for the scatterplot, so its code can be isolated in a reusable object. In the html, you have your svg defined for the scatterplot. In the scatterplot.js class, you create scales, create axes using these scales, append circles objects to your svg that

use the same scales, and encode the data - with hours to walk the trail encoded as x position, and trail distance encoded as y position, and trail difficulty encoded with color. After doing all that, there are 2 steps to adding in the ability to show data on hovering through a **tooltip**.

1. Setup: In the html file, you create a 'div', which is an html section to which you will add tooltip information. Let's call give it an identifier 'tooltip'. For now, it has nothing in it. and it won't show anything when the visualization first opens. Look below for: `<div id="tooltip"></div>`

```
<!DOCTYPE html>
<html>
<head>
  <title>D3 Interactive Scatter Plot</title>
  <link rel="icon" href="data:;base64,iVBORwOKGO=" />
  <link rel="stylesheet" href="css/style.css">
  <link
href="https://fonts.googleapis.com/css2?family=Noto+Sans+JP:wght@500;700&display=swap" rel="stylesheet">
</head>
<body>
  <h1>Vancouver Trails</h1>

  <svg id="scatterplot"></svg>

  <ul class="legend">
    <li class="legend-btn" data-difficulty="Easy"><span
class="legend-symbol"></span> Easy</li>
    <li class="legend-btn" data-difficulty="Intermediate"><span
class="legend-symbol"></span> Intermediate</li>
    <li class="legend-btn" data-difficulty="Difficult"><span
class="legend-symbol"></span> Difficult</li>
  </ul>

  <div class="source">
    Source: Data from <a
href="https://vancouvertrails.com/trails/">vancouvertrails.com</a>
  </div>

  <div id="tooltip"></div>

  <script src="js/d3.v6.min.js"></script>
  <script src="js/scatterplot.js"></script>
  <script src="js/main.js"></script>
</body>
```

```
</html>
```

2. Now, you need an **event listener** on the circles in your scatterplot. In html and svg, you can attach functions that activate when a particular interaction happens- if a user clicks, or mouses over, for instance. You can attach an event listener to svg elements on your page, and it will call the function you provide when that event is called, giving you both details on the event itself as well as what element the event was detected on. Here is how this works (this is after you have set up your whole plot).

```
// Tooltip event listeners
vis.circles
  .on('mouseover', (event,d) => {
    d3.select('#tooltip')
      .style('display', 'block')
      .style('left', (event.pageX + vis.config.tooltipPadding) + 'px')
      .style('top', (event.pageY + vis.config.tooltipPadding) + 'px')
      .html(`
        <div class="tooltip-title">${d.trail}</div>
        <div><i>${d.region}</i></div>
        <ul>
          <li>${d.distance} km, ~${d.time} hours</li>
          <li>${d.difficulty}</li>
          <li>${d.season}</li>
        </ul>
      `);
  })
  .on('mouseleave', () => {
    d3.select('#tooltip').style('display', 'none');
  });
```

Let's unpack this. Much of the above code concerns the style and format of the tooltip. But, the listener sends you event information and 'd' which is the element that is moused over. D3 has bound a data element to the circle, so you can use the data to specify what text to show in the tooltip. Html has list elements, which format text in a list. You can use string literals (`\${text\_to\_add}`) to add in the fields you want to print to the list elements.

Then, on 'mouse leave', which is called when the mouse exits the area of the svg element, you change 'display' to 'none' and you can't see the tooltip anymore.

Note: In this example, there are also style rules for the tooltip in the style sheet, that you can take note of. If you like the look of these tool tips, this is generated in the style

sheet (style/style.css), you can reuse these rules, or tweak them to suit your application. Here is what those rules look like.

```
/* in style/style.css */
/* Tooltip */
#tooltip {
  position: absolute;
  display: none;
  background: #fff;
  box-shadow: 3px 3px 3px 0px rgb(92 92 92 / 0.5);
  border: 1px solid #ddd;
  font-size: 12px;
  font-weight: 500;
  padding: 8px;
  min-width: 160px;
  color: #333;
}
.tooltip-title {
  color: #000;
  font-size: 14px;
  font-weight: 600;
}
#tooltip ul {
  margin: 5px 0 0 0;
  padding-left: 20px;
}
```

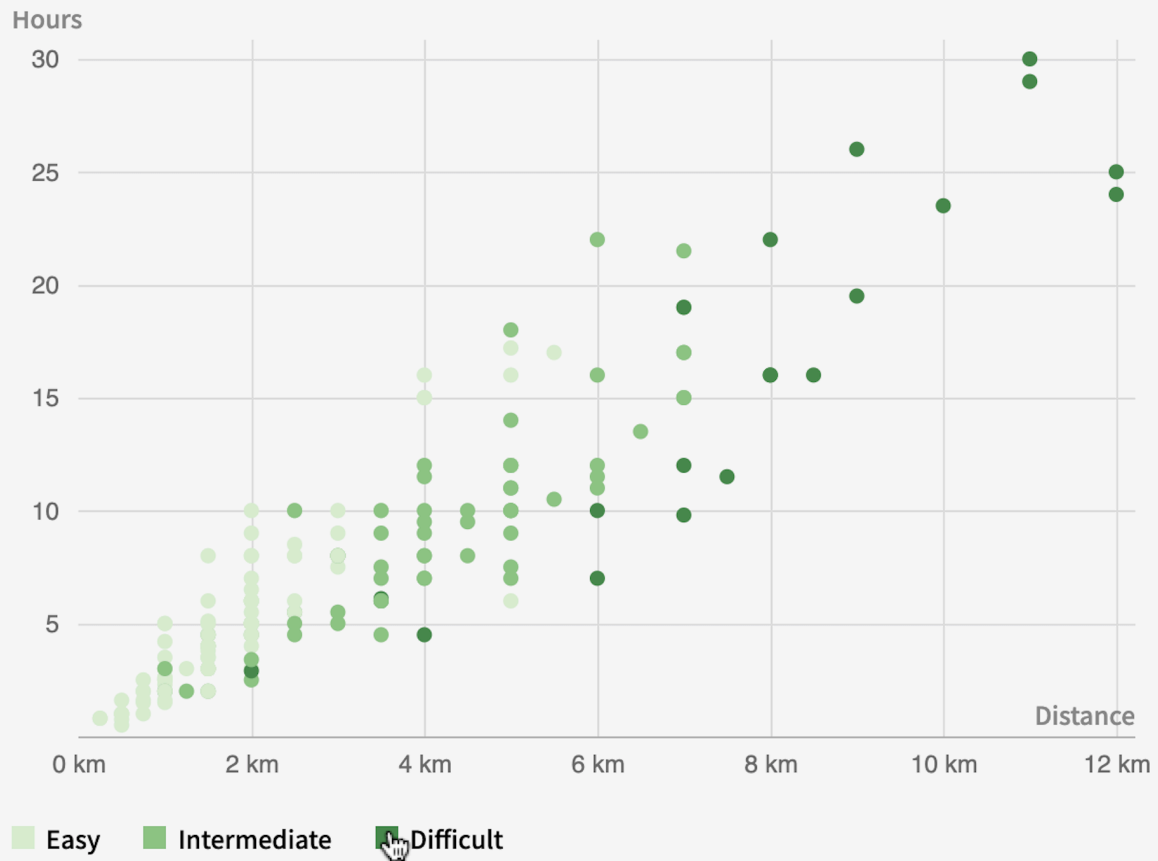
2. Filtering

What if you allow the user to make a selection that applies filter criteria to the data, so that fewer or more elements are shown. This means that we need a way to update the svg - adding or removing circles- to account for the change in the data. We don't want to redraw the visualization completely each time the data changes, but we just want to update the relevant components.

Here is an example- suppose you want to use legend buttons to allow users to select or deselect trails of different difficulty levels.

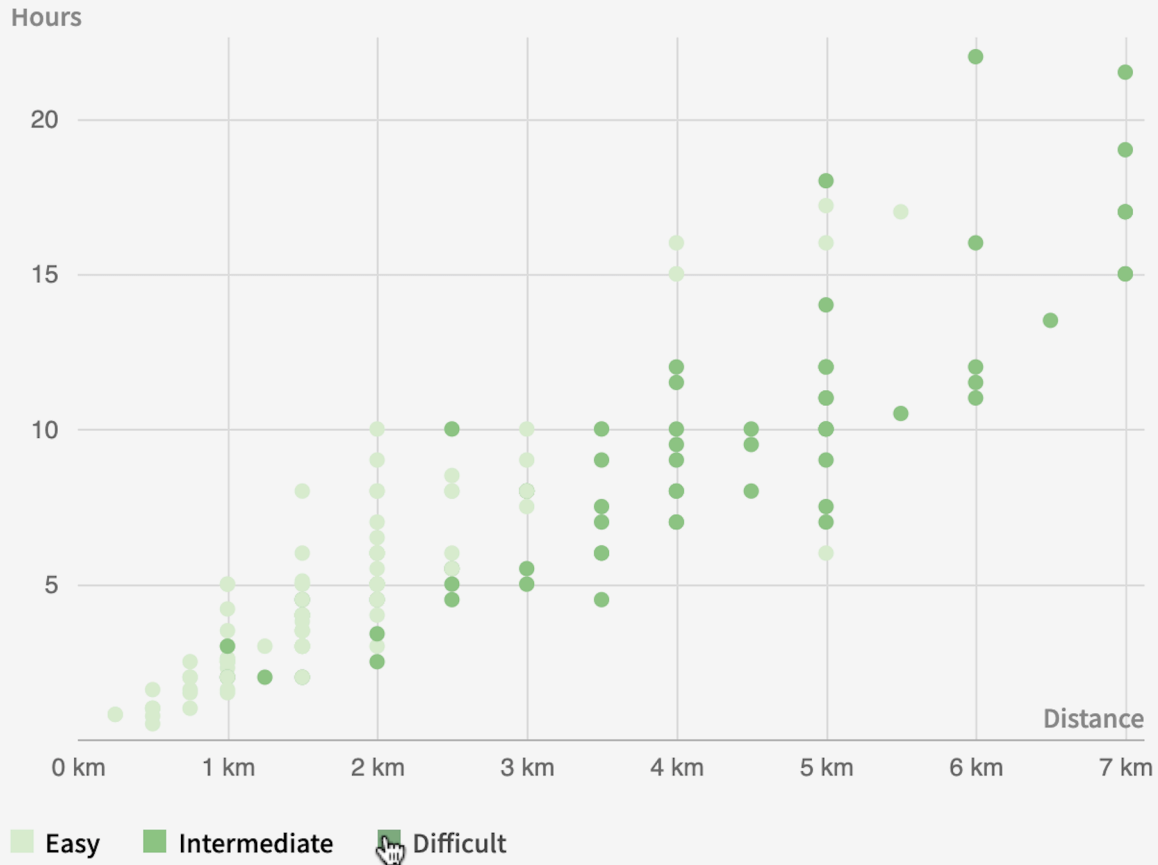
Before filtering out advanced trails

Vancouver Trails



After filtering out advanced trails- the advanced points are removed and the scales update.

Vancouver Trails



Source: Data from vancouvertrails.com

Setup of buttons and event handling:

First of all, how are these buttons at the bottom created? You can add UI elements- like buttons, checkboxes, sliders, drop down menus- in the html. These three buttons are given a common class label "legend-btn".

```
<ul class="legend">

  <li class="legend-btn" data-difficulty="Easy"><span
class="legend-symbol"></span> Easy</li>

  <li class="legend-btn" data-difficulty="Intermediate"><span
class="legend-symbol"></span> Intermediate</li>

  <li class="legend-btn" data-difficulty="Difficult"><span
class="legend-symbol"></span> Difficult</li>
```


These buttons have style rules - color, size, rules for how the color changes when hovered over or when deselected. These style rules are set in style/style.css and look like this (below). You'll notice that they use the class selector ('.legend-btn') and also multi-class selectors ('.legend-btn:hover.legend-btn.inactive:hover'). The basic principle here is that when buttons are deactivated or hovered over, you want to apply a different look to the buttons, while still allowing them to remain in the 'legend-btn' class.

```
/* in style/style.css */
/* Legend */
.legend {
  margin: 20px 0;
  list-style: none;
  padding: 0;
}
.legend-btn {
  display: inline-block;
  margin: 0 20px 0 0;
  cursor: pointer;
  transition: all ease-in-out 200ms;
}
.legend-btn:hover.legend-btn.inactive:hover {
  opacity: 0.7;
}
.legend-btn.inactive {
  opacity: 0.5;
}
.legend-symbol {
  width: 12px;
  height: 12px;
  margin-right: 3px;
  display: inline-block;
}
.legend-btn[data-difficulty="Easy"] .legend-symbol {
  background: #d3eecd;
}
.legend-btn[data-difficulty="Intermediate"] .legend-symbol {
  background: #7bc77e;
}
.legend-btn[data-difficulty="Difficult"] .legend-symbol {
  background: #2a8d46;
}
```

Personally, I think their style rules are somewhat subtle, and it might be better to have more obvious changes on selection and deselection.

Event detection

In main.js, you can add an event listener to these buttons. This time it will detect a 'click' event and first it will toggle the checkbox from clicked to not clicked.

```
/**
 * Event listener: use color legend as filter
 */
d3.selectAll('.legend-btn').on('click', function() {
  // Toggle 'inactive' class
  d3.select(this).classed('inactive',
!d3.select(this).classed('inactive'));

  // Check which categories are active
  let selectedDifficulty = [];
  d3.selectAll('.legend-btn:not(.inactive)').each(function() {
    selectedDifficulty.push(d3.select(this).attr('data-difficulty'));
  });

  // Filter data accordingly and update vis
  scatterplot.data = data.filter(d =>
selectedDifficulty.includes(d.difficulty));
  scatterplot.updateVis();
});
```

Recall that graphical elements - like scatterplot circles- can be added through a chain of calls, which **appended** an element to the svg for all data elements and then called **'enter'** to finalize the action. Now, we need change those data elements- including removing some of them.

Instead of 'append' and 'enter', we are going to use a call to a function **'join'**. Join was added in version 5 of d3, so you won't see it in v4 of d3. Previously people used different calls on the chain for different actions: 'enter' to add, or 'update' to change or 'exit' to remote. You can still use this approach, but often we can just **use join to cover all 3 cases**.

See below how we use 'join'- which represents both adding new elements, updating previous elements that have some change in state, as well as removing elements no longer in the filtered data.

```
// Add circles, and use join - allows for adding, removing, updating
```

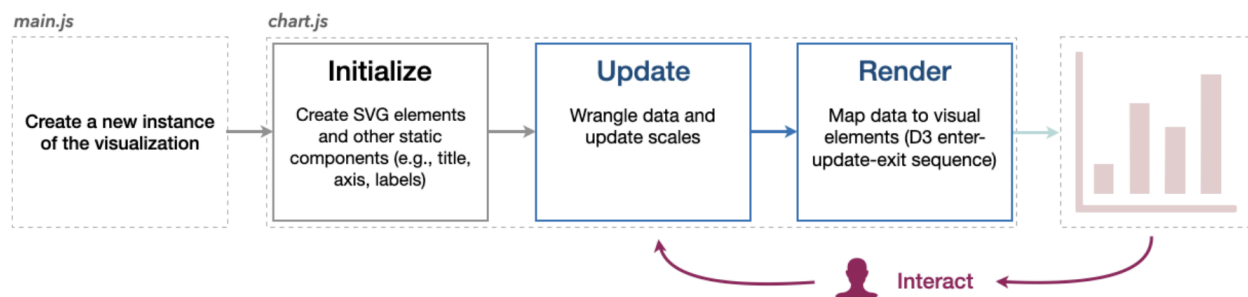
```
vis.circles = vis.chart.selectAll('.point')
  .data(vis.data, d => d.trail)
  .join('circle')
  .attr('class', 'point')
  .attr('r', 4)
  .attr('cy', d => vis.yScale(vis.yValue(d)))
  .attr('cx', d => vis.xScale(vis.xValue(d)))
  .attr('fill', d => vis.colorScale(vis.colorValue(d)));
```

This is a nice in-depth walk-through of join:

<https://www.d3indepth.com/datajoins/>

Code restructuring

We can't put all our code in the init function anymore. Init is only called once, after the constructor is called. Some of our code to draw elements need to move into a function that can be called both at the beginning AND when someone interacts. A common way to do this is through an 'update' function, that is called after 'init' AND on user events.



When someone clicks on a button to filter, we need to:

- update the data that is drawn
- and (in this scatterplot at least) update the axes, to reflect a potentially new maximum value

We can move the code we expect to need to call when the user acts into an 'update' function. Then we can call 'update'

At the moment, we don't need a separate update and render function, so we will keep some things in init and put things that update into the 'update' function.